



Neural Radiance Field!

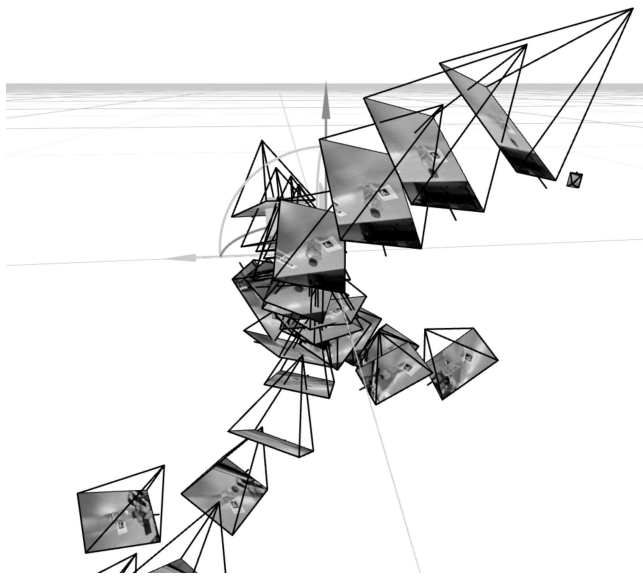
PART 0

Camera Calibration and 3D Scanning

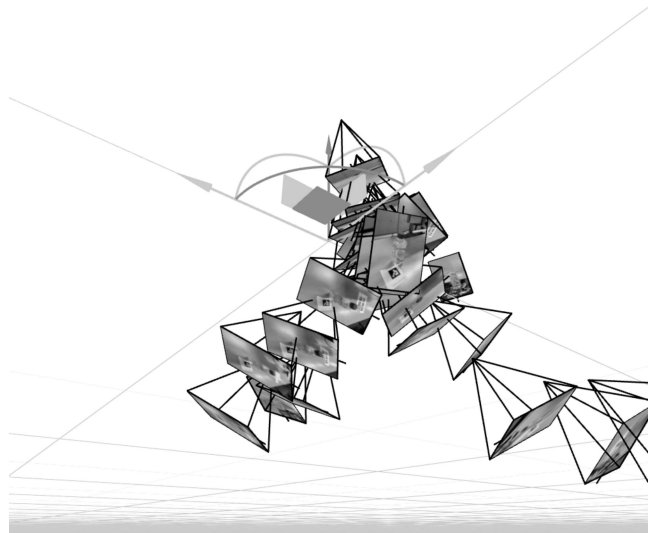
Viser Camera Frustums

I have used my iPhone 15 camera to capture the images of the object. I have used the `calibration.py` script to get the camera intrinsics and then used the `camera_pose.py` script to get the camera poses and to visualize the camera frustums.

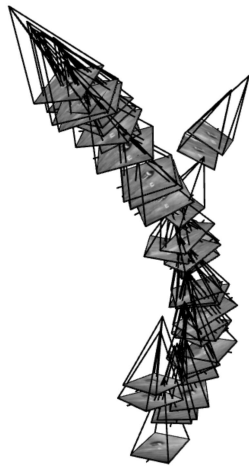
For the first try, I used a bottle. Two screenshots of camera frustums visualization in Viser are shown below. As they were obviously not correct and I could not find any bug in the code, I have decided to use a different object. The camera poses for the football object are shown below. Again, there seems to be a mistake, but even after extensive debugging, I could not find any bug in the code. Lastly, I tried the `lafufu` data (incl. calibration dataset). However, I could not manage to get the right camera poses (see below).



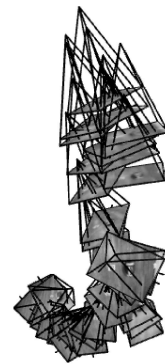
Bottle - Frustums View 1



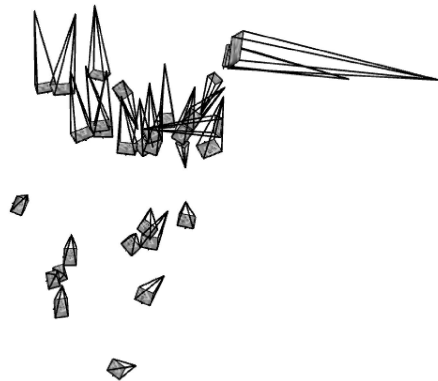
Bottle - Frustums View 2



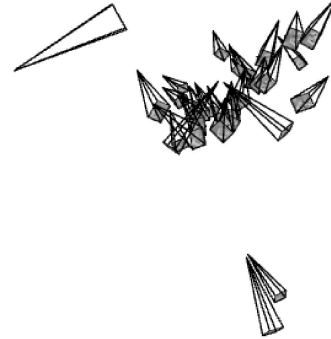
Football - Frustums View 1



Football - Frustums View 2



Lafufu - Frustums View 1



Lafufu - Frustums View 2

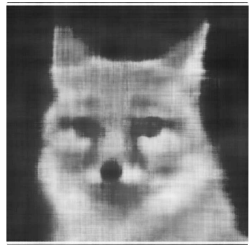
PART 1

Fit a Neural Field to a 2D Image

Model Architecture Report

- **Layers:** 1 positional encoding module (PosEnc, $L=10$);
4-layer MLP:
 - Linear($pe_dim \rightarrow 256$) $\rightarrow$ ReLU,
 - Linear($256 \rightarrow 256$) $\rightarrow$ ReLU,
 - Linear($256 \rightarrow 256$) $\rightarrow$ ReLU,
 - Linear($256 \rightarrow 3$) $\rightarrow$ Sigmoid
- **Width:** 256 neurons per hidden layer
- **Learning Rate:** $1e-2$ (Adam optimizer)
- **Other Details:**
 - Activation: ReLU (hidden layers), Sigmoid (output layer)
 - Positional Encoding: $L=10$ (uses 10 frequency bands as input PE)
 - $pe_dim = 2 \text{ input} + 2 \cdot (2 \cdot L) = 42$
 - Output: 3 channels (RGB), normalized $[0,1]$

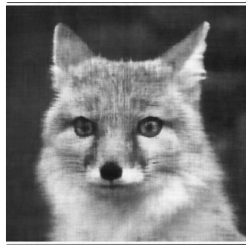
Training Progression



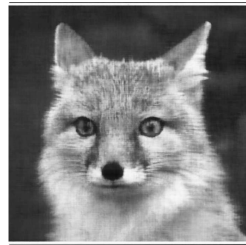
Wolf, 100 iters



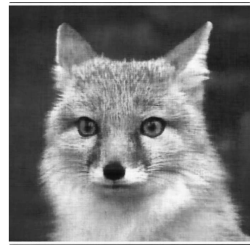
Wolf, 200 iters



Wolf, 500 iters



Wolf, 1000 iters



Wolf, 1500 iters

Training Progression (Kite)



Kite, 100 iters



Kite, 200 iters



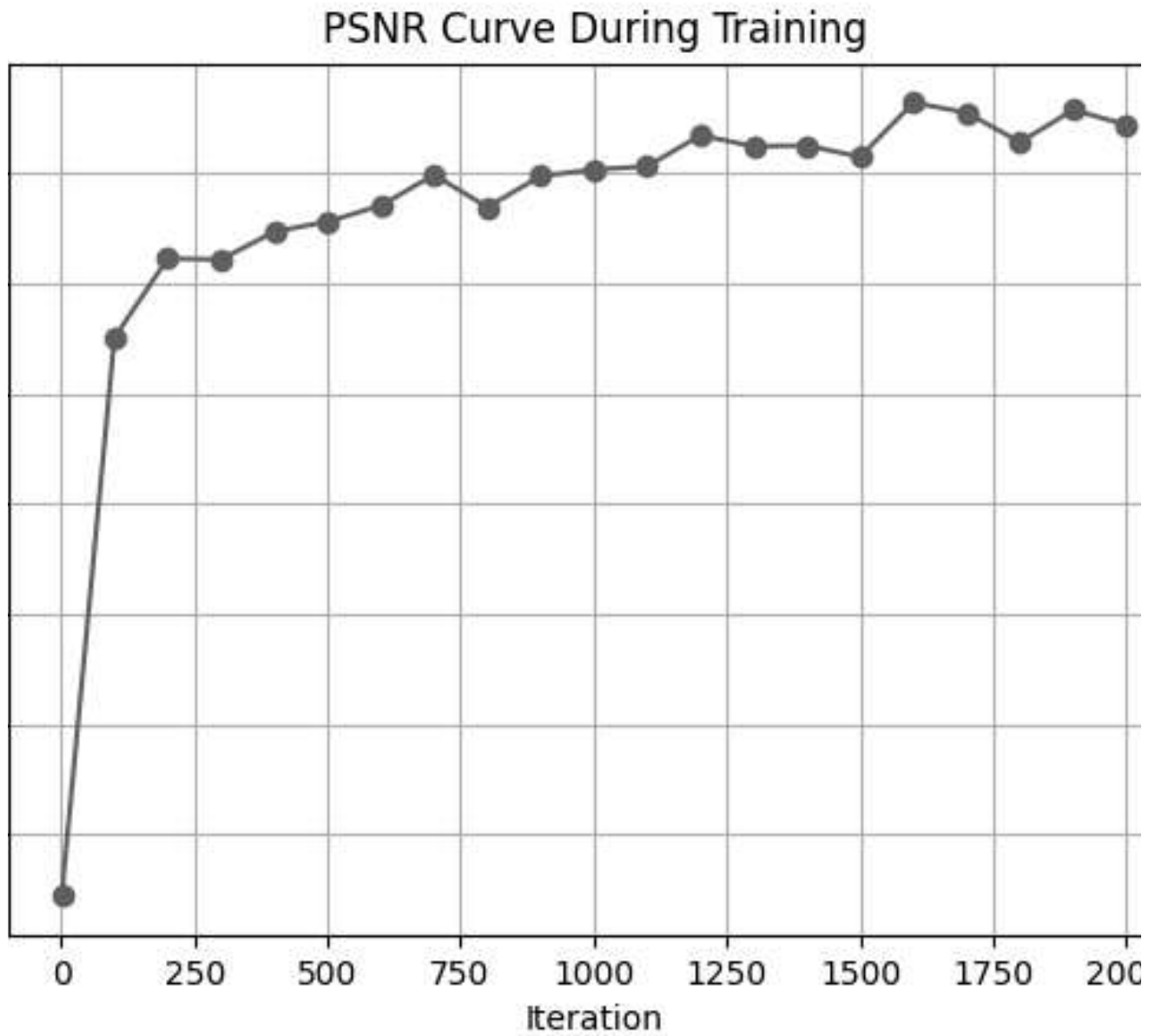
Kite, 500 iters



Kite, 1000 iters



Kite, 1500 iters



PSNR curve for Kite

Final Results (2×2: PE max freq × width)



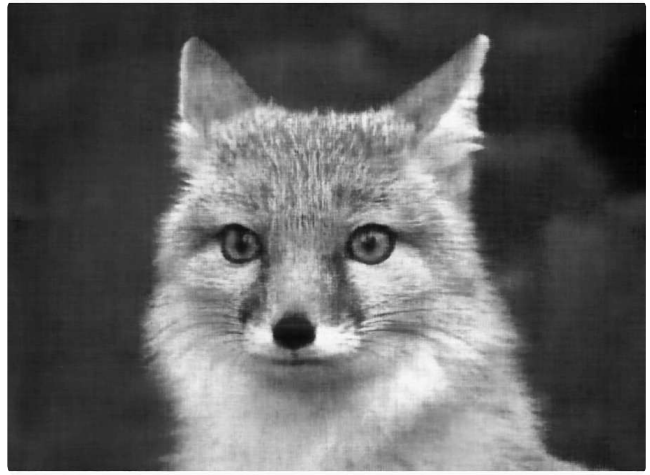
PE 2 × Width 64



PE 2 × Width 256



PE 10 × Width 64



PE 10 × Width 256

As one can see, the higher PE frequency allows for higher frequencies to show in the final results, and a higher width results in smoother and more detailed results, albeit at the cost of lacking high frequencies with $L=2$.

PART 2

Fit a Neural Radiance Field from Multi-view Images

Implementation Notes

Part 2.1

Camera to World Coordinate Conversion

The camera to world coordinate conversion is implemented in the `transform_c2w` function in the `nerf_multi_view.py` file. The function takes a camera to world coordinate matrix and a point in the camera coordinate system and returns the point in the world coordinate system. It appends a `[1]` to the point and then multiplies it by the camera to world coordinate matrix using `einsum` to get the point in the world coordinate system.

Pixel to Camera Coordinate Conversion

The pixel to camera coordinate conversion is implemented in the `pixel_to_camera` function in the `nerf_multi_view.py` file. It multiplies the point by the inverse of the camera intrinsics matrix using `einsum` to get the `u, v` point in the camera coordinate system.

Pixel to Ray Conversion

The pixel to ray conversion is implemented in the `pixel_to_ray` function in the `nerf_multi_view.py` file. It implements the conversion using the given formula.

Part 2.2

Sampling Rays from Images and Sampling Points Along Rays

The sampling rays from images is implemented in the `sample_random_rays` function in the `nerf_multi_view.py` file. It samples rays from the images using the `pixel_to_ray` function and returns the origin and direction along with the colors and indices. The origins and directions are used in the `sample_points_along_rays` function to generate the points along the rays (64 samples per ray starting from 2.0 to 6.0 in the near and far plane).

Part 2.3

RaysData Class

The `RaysData` class is implemented in the `nerf_multi_view.py` file. It contains the images, camera intrinsics matrix and camera to world coordinate matrices. It also contains the function to sample

rays from the images.

Part 2.4

Neural Field Training

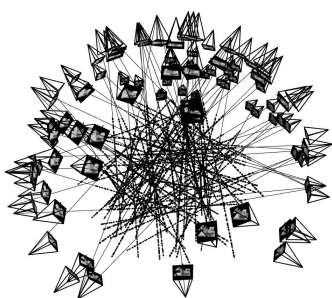
The neural field training is implemented in the `train_nerf` function in the `nerf_neural_field_3d.py` file. It trains the neural field that is provided in the task description using the data sampled by the `RaysData` class. It can save and load checkpoints to resume training.

Part 2.5

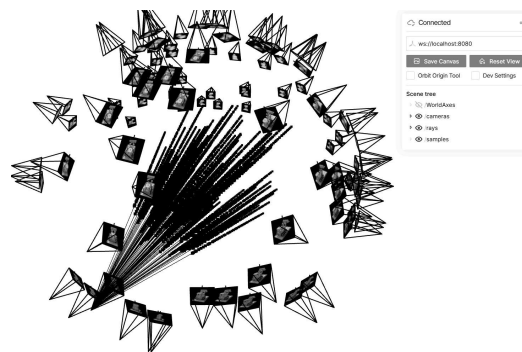
Volume Rendering

The volume rendering step is implemented in the `volrend` function in `volume_rendering.py`. This function computes opacities and transmittance along sampled points on rays. It combines them with the predicted colors to calculate the final rendered color using the provided formula.

Rays and Samples Visualization



Rays and samples with cameras (View 1)

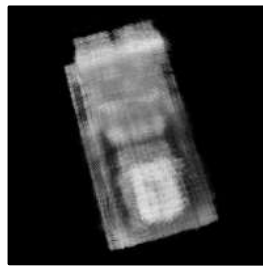


Rays and samples with cameras (View 2)

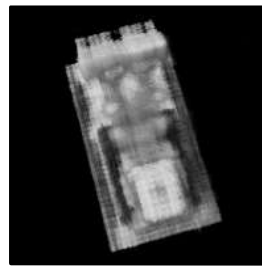
Training Progression (Predicted Images)



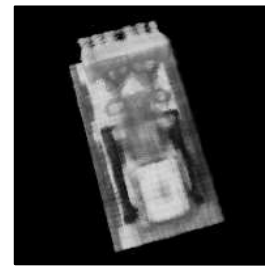
Iter 100



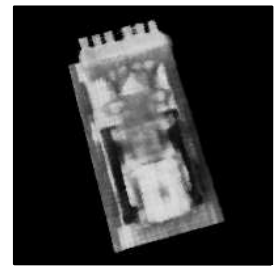
Iter 500



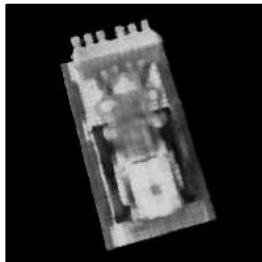
Iter 1000



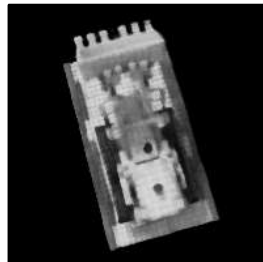
Iter 1500



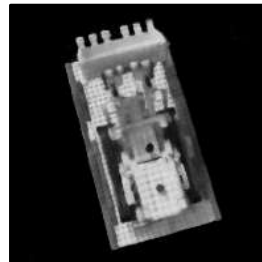
Iter 2000



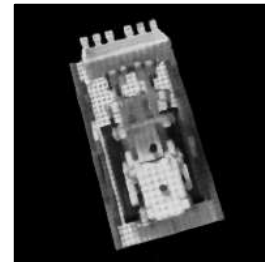
Iter 2500



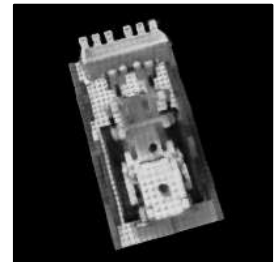
Iter 5000



Iter 10000

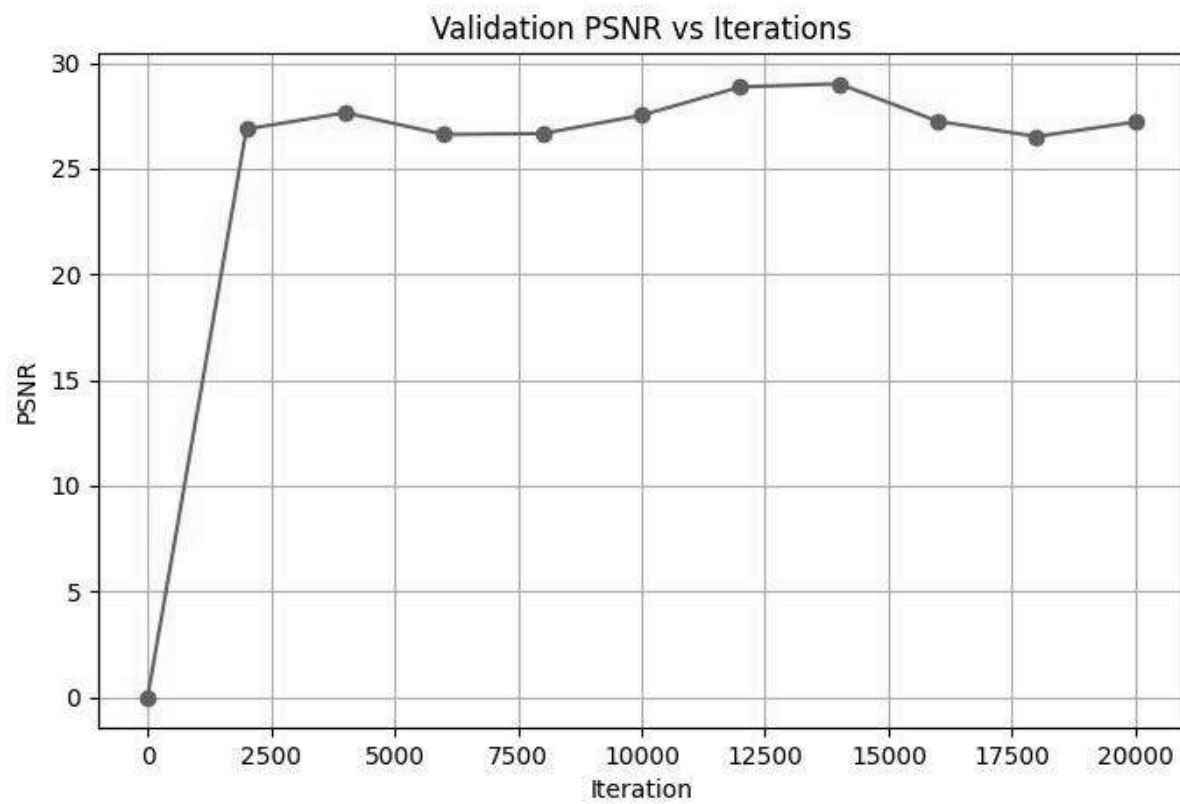


Iter 15000



Iter 20000

PSNR on Validation Set



PSNR vs. iterations (validation)

Spherical Rendering Video (Lego)

0:00 / 0:06



Spherical render

PART 2.6

Training with Your Own Data

I tried using the camera poses generated in Part 0 for the bottle, football and lafufu data. However, I could not manage to train meaningful results on any of the objects and the resulting images stayed gray. Due to time constraints, I have not been able to debug the issue and I have not been able to train the neural field on any of the objects.

© 2025 — Konstantin Wehmeyer